CRACKING THE

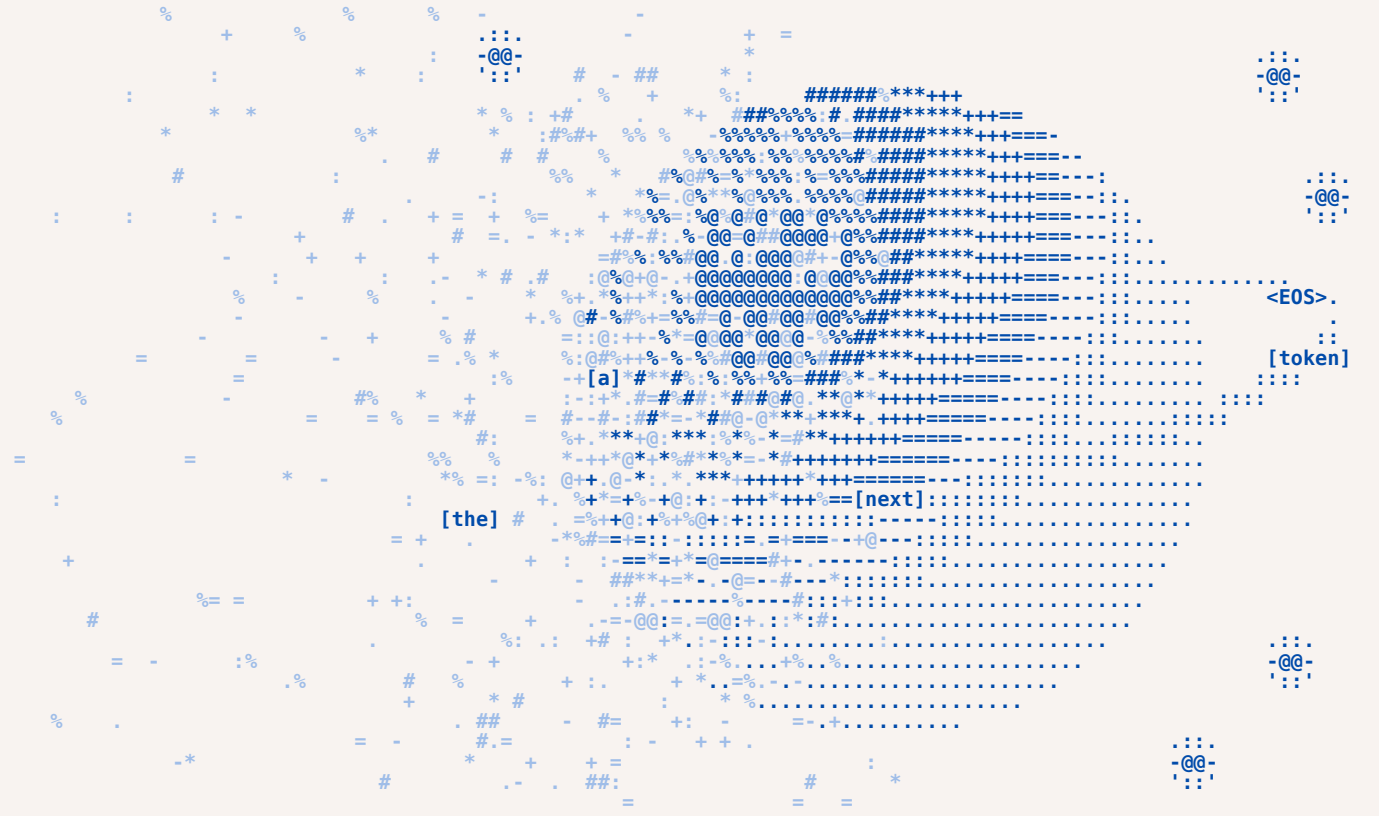
```
  ,ad8888ba,  88888888888 888b      88 88888888888 88888888ba         db   888888888888 88 8b           d8 88888888888
 d8"'    `"8b 88          8888b     88 88          88      "8b       d88b       88      88 `8b         d8' 88
d8'          88          88 `8b    88 88          88      ,8P      d8'`8b      88      88  `8b       d8'  88
88           88aaaaa     88  `8b   88 88aaaaa     88aaaaaa8P'     d8'  `8b     88      88   `8b     d8'   88aaaaa
88      88888 88"""""     88   `8b  88 88"""""     88""""88'      d8YaaaaY8b    88      88    `8b   d8'    88"""""
Y8,        88 88          88    `8b 88 88          88    `8b     d8""""""""8b   88      88     `8b d8'     88
 Y8a.    .a88 88          88     `8888 88          88     `8b   d8'        `8b  88      88      `888'      88
  `"Y88888P"  88888888888 88      `888 88888888888 88      `8b d8'          `8b 88      88       `8'       88888888888
```

```
      db          88
     d88b         88
    d8'`8b        88
   d8'  `8b       88
  d8YaaaaY8b      88
 d8""""""""8b     88
d8'          `8b  88
d8'            `8b 88
```

ENGINEERING INTERVIEW 2026

The Essential Cheat Sheet for LLMs, RAG, AI Agents, Multimodal AI, Fine-Tuning, and Production AI Systems

$ ./generate --from noise --to answer x_T ~ N(0, I) ----- denoise -----> x_0



[01] Foundations of Generative AI
[02] Transformer & LLM Architecture
[03] Prompt Engineering
[04] LLM Inference & Serving
[05] Retrieval-Augmented Generation
[06] AI Agents
[07] Multimodal AI
[08] Fine-Tuning & Model Adaptation
[09] Production Generative AI Systems
[10] Evaluation & System Design

BUILD STATUS [##] 100% 100 Q&A / 3 MOCK INTERVIEWS / CHEAT SHEET

Cracking the Generative AI Engineering Interview 2026

The Essential Cheat Sheet for LLMs, RAG, AI Agents, Multimodal AI, Fine-Tuning, and Production AI Systems

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

First edition, 2026. Published by AI Engineering Insider.

Web: aiengineeringinsider.com Newsletter: aiengineeringinsider.substack.com/subscribe

Disclaimer. The information in this book is provided on an "as is" basis, without warranty of any kind. Product names, logos, and brands—including OpenAI, Anthropic, Google, Meta, Llama, DeepSeek, Qwen, Mistral, NVIDIA, vLLM, SGLang, TensorRT-LLM, llama.cpp, Ollama, LangChain, LangGraph, CrewAI, AutoGen, Hugging Face, Pinecone, Qdrant, Weaviate, Milvus, and PostgreSQL—are the property of their respective owners and are used for identification purposes only; their use does not imply endorsement. This book is an independent educational work and is not affiliated with or endorsed by any model provider, framework maintainer, or standards body named within.

PREFACE

A language model reads a token and predicts a token. Everything else in a generative AI system — the tokenizer, the retriever, the reranker, the KV cache, the agent loop, the guardrail — exists to decide *which* tokens it reads, and to make the reading cheap enough that the product survives contact with traffic.

That is not how generative AI is usually taught, and it is not how most interviews are prepared for. The common preparation is a vocabulary exercise: know what RAG stands for, know that LoRA is parameter-efficient, know that agents use tools. That vocabulary gets you through a screening call and collapses in the second round, because the questions that actually distinguish candidates are not definitional. They are: *how large is the KV cache for this model at this context, and what does that mean for your batch size? Your retrieval returns semantically similar but factually wrong passages — is a better embedding model the fix? Your agent works in testing and loops in production — walk me through the debugging.*

Those questions have real answers, and the answers are engineering rather than trivia. This book is organised around producing them. Each chapter builds the mental model for one layer of the stack, states the trade-offs where decisions actually branch, and then works through ten interview questions with answers at the depth a senior interview expects — system design, production operations, debugging, technical mechanism, and case studies drawn from the failure modes that recur across real deployments.

The perspective throughout is a system designer's. Research questions ask which method is better. Production questions ask which method is better *at this scale, on this traffic mix, under this latency SLO, with this failure tolerance* — and the answer changes with all four. Grouped-query attention is not better than multi-head attention; it is the configuration that makes a 70B model economically servable. Retrieval-augmented generation is not better than a long context window; it is what you use when the corpus updates, when answers need citations, and when a customer can ask you to delete their data. Chapter after chapter, the argument is not that one approach wins, but that you should be able to say which one wins for you, and show the number.

More practitioner deep dives: https://aiengineeringinsider.substack.com/subscribe

AI Engineering Insider
2026

ABOUT THIS BOOK

+==+

This book is a system-design-level interview guide to generative AI: how text becomes tokens, how tokens become answers, how those answers are grounded, served, secured, evaluated, and paid for. It is written for AI engineers, machine learning engineers, inference and ML systems engineers, agentic-AI and forward-deployed engineers, software engineers moving into AI, and AI architects who are past the tutorials and need the patterns, trade-offs, failure modes, and runnable code that show up in production and in senior interviews.

» HOW THIS VOLUME IS ORGANISED

Ch.	Title	Main focus
1	Foundations of Generative AI	Discriminative versus generative modelling and the chain-rule factorisation; foundation-model families; self-attention, FFN, residuals, and normalisation placement; tokenization algorithms and the real cost of non-English text; embedding types and what cosine similarity cannot represent; scaling laws, Chinchilla optimality versus total cost of ownership, and emergent abilities.
2	Transformer and LLM Architecture	The modern decoder block and residual stream; MHA, MQA, GQA, and MLA as KV-cache economics; RoPE, ALiBi, and context extension; the KV-cache sizing formula and PagedAttention; Mixture-of-Experts and what it actually trades; FlashAttention and the roofline view of prefill versus decode.
3	Prompt Engineering	Prompt anatomy and the trust boundary; system-prompt design and instruction dilution; zero-shot, few-shot, and dynamic example selection; chain-of-thought and when it hurts; structured output and constrained decoding; prompt optimisation as a measured loop; injection, jailbreaks, leakage, and layered defence.
4	LLM Inference and Serving	The generation pipeline end to end; sampling methods and their production settings; continuous batching and chunked prefill; prefix caching as the highest-leverage optimisation; speculative decoding and where it backfires; vLLM, SGLang, TensorRT-LLM, and llama.cpp; serving topology, admission control, and autoscaling signals.

continued on the next page

Ch.	Title	Main focus
5	Retrieval-Augmented Generation	RAG against fine-tuning and long context; parsing and the failures that cap quality; chunking strategies and parent–child retrieval; dense, sparse, and hybrid retrieval with reciprocal rank fusion; vector stores and ANN index trade-offs; bi-encoders versus cross-encoders; query transformation; component-wise evaluation and error attribution.
6	AI Agents	The autonomy spectrum and why reliability compounds multiplicatively; agent components and tool design; memory types, provenance, and poisoning; context management and the structured scratchpad; multi-agent patterns and when they are justified; human-in-the-loop by reversibility; trajectory evaluation and cost per successful task.
7	Multimodal AI	The token cost of every modality; vision-language architecture and the connector’s accuracy cliff; dynamic tiling and resolution as capability; OCR versus VLM extraction and silent transcription hallucination; ASR, TTS, and the voice-latency budget; diffusion and controllability; multimodal RAG and video sampling policy.
8	Fine-Tuning and Model Adaptation	Form versus fact as the decision criterion; SFT mechanics and loss masking; LoRA mathematics, target modules, and rank selection; QLoRA and the memory ladder; RLHF, DPO, IPO, KTO, and ORPO; dataset quality as the dominant variable; synthetic data and model collapse; catastrophic forgetting and capability gating.
9	Production Generative AI Systems	The reference architecture; API design for an expensive non-deterministic dependency; layered guardrails with explicit failure policy; the caching hierarchy and semantic-cache risk; token-based rate limiting; observability and cost attribution; Kubernetes deployment, cold starts, and autoscaling on the right signals.
10	Evaluation, Optimisation, and System Design	Evaluation methods and their limits; LLM-as-judge calibration and bias control; benchmark contamination; the performance and cost optimisation ladders ordered by gain per unit of risk; structural hallucination detection; security and responsible AI; and a repeatable framework for generative AI system design interviews.

» WHAT IS NEW IN THIS EDITION

This edition adds the material below.

* **Cheat sheet.** Twelve reference blocks after the contents: formulas, worked numbers, and decision tables.
* **2026 updates.** New sections on reasoning models and test-time compute, Blackwell FP4 serving, and provider prompt caching (Chapter 4), GraphRAG and agentic RAG (Chapter 5), and the Model Context Protocol and agent sandboxes

(Chapter 6). A yellow [2026 UPDATE] tag marks each one.

* **Mock interviews.** Three transcripts with coach notes, at the end of Chapters 4, 5, and 6.
* **Glossary.** Definitions of 151 terms at the end of the book.
* **Corrections.** The KV cache values in Chapter 2 and the speculative decoding example in Chapter 4 were calculated again.

CONTENTS

Preface i

About This Book ii

Cheat Sheet xi

PART I Foundations 1

[1] Foundations of Generative AI 2

» 1.1 Overview 2

» 1.2 Discriminative versus Generative Models 4

» 1.3 Foundation Models 5

» 1.4 Neural Network Fundamentals for the Transformer 6

» 1.5 Tokenization 8

» 1.6 Embeddings 10

» 1.7 Scaling Laws and Emergent Abilities 11

» 1.8 The Ecosystem: Open versus Closed 12

» 1.9 Capabilities and Limitations 13

» 1.10 Interview Questions and Answers 14

Q01.01 Explain the difference between discriminative and generative models, and describe a production system where you would deliberately use both. 14

Q01.02 Your team estimated context costs using characters divided by four. After launching in Japan, users report truncated answers and your bill is 2.5x forecast. Diagnose and fix. 15

Q01.03 Why does the attention formula divide by $\sqrt{d_k}$? What breaks if you remove it? 16

Q01.04 Compare Pre-LN and Post-LN Transformers. Which would you pick for a 70B model and why? 17

Q01.05 A stakeholder asks: "Models now have 1M-token context—can we delete our RAG pipeline?" Give your recommendation with reasoning. 18

Q01.06 Explain scaling laws and Chinchilla optimality. Your company can spend a fixed compute budget—how do you allocate it, and does the answer change if the model will serve production traffic for two years? 18

Q01.07 What are emergent abilities, and how would you respond to a colleague who claims they prove models will suddenly become dangerous at some scale? . . 20

Q01.08 Your RAG system returns semantically similar but factually wrong passages. A colleague proposes a better embedding model. Is that the right fix? . . . 20

Q01.09 Walk through what happens between a user pressing Enter and the first token appearing on screen. 22

Q01.10 Design the model-selection strategy for a startup with three workloads: high-volume classification, customer-facing chat, and a low-volume legal analysis tool. 23

[2] Transformer and LLM Architecture 26

» 2.1 Overview 26

» 2.2 The Transformer Block 27

» 2.3 Attention Variants . . . 28
» 2.4 Positional Encoding . . . 29
» 2.5 The KV Cache . . . 31
» 2.6 Mixture of Experts . . . 33
» 2.7 Efficient Attention Kernels . . . 34
» 2.8 Long Context . . . 35
» 2.9 Interview Questions and Answers . . . 36
Q02.01 Derive the KV cache size for Llama-3-70B at 8k context and explain what it means for your serving architecture. . . . 36
Q02.02 Explain MHA, MQA, GQA, and MLA. Which would you choose for a 30B model serving 5k concurrent chat users? . . . 37
Q02.03 Your team fine-tuned a model with RoPE base scaling to extend context from 8k to 64k. Benchmarks look fine but users report the model "forgets" material in long documents. Debug this. . . . 37
Q02.04 Why is FlashAttention faster if it performs the same number of FLOPs? . . . 39
Q02.05 A colleague proposes switching from a dense 70B model to a 400B-total / 40B-active MoE to "get better quality at the same cost." Evaluate this. . . . 40
Q02.06 Explain the KV cache to a backend engineer with no ML background, then explain why it causes out-of-memory errors under load. . . . 41
Q02.07 Why are LLM outputs non-deterministic even at temperature 0? Your legal client requires reproducibility—what do you tell them? . . . 42
Q02.08 Design the attention configuration for a code-completion model that must serve 50ms p99 latency with 32k context. . . . 44
Q02.09 What is the difference between the prefill and decode phases, and why does it dictate your entire serving architecture? . . . 45
Q02.10 You must serve a 7B model on a single 24GB consumer GPU with 8k context and maximum concurrency. Walk through your memory budget and optimisations. 46

PART II Building Blocks . . . 50

[3] Prompt Engineering . . . 51
» 3.1 Overview . . . 51
» 3.2 Prompt Anatomy . . . 52
» 3.3 System Prompt Design . . . 53
» 3.4 Zero-shot, Few-shot, and Example Selection . . . 54
» 3.5 Chain-of-Thought and Reasoning . . . 55
» 3.6 Structured Output . . . 56
» 3.7 Prompt Optimisation . . . 58
» 3.8 Prompt Security . . . 58
» 3.9 Interview Questions and Answers . . . 60
Q03.01 Your prompt works in testing and fails in production 15% of the time. Walk through your debugging process. . . . 61
Q03.02 Explain prompt injection and design a defence for a RAG system that ingests customer-uploaded PDFs. . . . 63
Q03.03 When should you use few-shot prompting versus fine-tuning? Give the decision criteria and the crossover math. . . . 65
Q03.04 A support bot leaks its system prompt when users ask cleverly. How serious is this and what do you do? . . . 66
Q03.05 Design a prompt system for a multi-tenant SaaS where each customer needs custom behaviour but you must control safety and cost. . . . 67
Q03.06 Explain chain-of-thought prompting. When does it hurt, and how would you decide whether to use it? . . . 69
Q03.07 Your extraction pipeline returns malformed JSON about 3% of the time and you have a retry loop. Critique this design. . . . 71
Q03.08 Design an A/B testing framework for prompt changes in production. . . . 72
Q03.09 How do you handle prompts that must work across multiple model providers? . 74

Q03.10 A prompt that worked for months suddenly degrades after a provider model update. Walk through your response. 76

[4] LLM Inference and Serving 79

» 4.1 Overview 79
» 4.2 The Text Generation Pipeline 81
» 4.3 Sampling Methods 81
» 4.4 Continuous Batching 84
» 4.5 Prefix Caching 85
» 4.6 Speculative Decoding 86
» 4.7 Inference Frameworks 88
» 4.8 Serving Architecture 89
» 4.9 Reasoning Models and Test-Time Compute 89
» 4.10 Serving on Blackwell: FP8 and FP4 91
» 4.11 Provider Prompt Caching 92
» 4.12 Interview Questions and Answers 93

Q04.01 Explain continuous batching and quantify the improvement over static batching. 93
Q04.02 Your service has p50 TTFT of 200ms but p99 of 8 seconds. Diagnose. 94
Q04.03 Explain PagedAttention and why it was a breakthrough. 95
Q04.04 What is speculative decoding, when does it help, and when does it hurt? . . 97
Q04.05 Design an LLM serving system for 10,000 requests/second with mixed workloads. 98
Q04.06 Your inference cost is $180k/month. The CFO wants it halved without quality loss. What do you do? 100
Q04.07 How do you handle streaming responses, and what breaks in production? . . . 102
Q04.08 Compare vLLM, TensorRT-LLM, and llama.cpp. When would you choose each? . . 104
Q04.09 Your GPU fleet OOMs during traffic spikes despite 30% headroom at steady state. Explain and fix. 105
Q04.10 Explain the memory-bandwidth bound in decode and what actually helps. . . . 107

» 4.13 Mock Interview: Size a Reasoning Endpoint 109

[5] Retrieval-Augmented Generation 112

» 5.1 Overview 112
» 5.2 Document Parsing 114
» 5.3 Chunking 115
» 5.4 Retrieval 116
» 5.5 Reranking 118
» 5.6 Query Transformation 119
» 5.7 Evaluation 120
» 5.8 GraphRAG and Agentic RAG 121
» 5.9 Interview Questions and Answers 123

Q05.01 Design a RAG system for 50 million enterprise documents with per-user access control. 123
Q05.02 Users say your RAG chatbot gives correct answers to first questions but nonsense on follow-ups. Diagnose. 124
Q05.03 Explain the difference between bi-encoders and cross-encoders and why production systems use both. 126
Q05.04 Your RAG system's answers are technically grounded but users say they "miss the point." Investigate. 128
Q05.05 How would you evaluate a RAG system before and after launch? 129
Q05.06 Compare chunking strategies. How would you choose for a corpus of legal contracts? 131
Q05.07 Your retrieval latency is 800ms p99 and the budget is 200ms. Optimise. . . 133

Q05.08 When would you NOT use RAG? . . . 136
Q05.09 Design a RAG evaluation and monitoring system that detects quality degradation before users complain. . . . 137
Q05.10 You must migrate a 200-million-chunk index to a new embedding model with zero downtime. Design the migration. . . . 139
» 5.10 Mock Interview: Retrieval for a Contract Corpus . . . 141

PART III Advanced Systems . . . 143

[6] AI Agents . . . 144
» 6.1 Overview . . . 144
» 6.2 Agent Components . . . 146
» 6.3 Agent Memory . . . 148
» 6.4 Context Management: The Real Constraint . . . 149
» 6.5 Multi-Agent Systems . . . 150
» 6.6 Human in the Loop . . . 150
» 6.7 Agent Frameworks . . . 151
» 6.8 Agent Evaluation . . . 151
» 6.9 The Model Context Protocol (MCP) . . . 152
» 6.10 Sandboxing Agent Actions . . . 153
» 6.11 Interview Questions and Answers . . . 154
Q06.01 Design an agent that resolves customer support tickets end to end. Cover architecture, tools, safety, and evaluation. . . . 154
Q06.02 Your agent works in testing but loops infinitely in production. Debug it. . . 156
Q06.03 When should you use a multi-agent architecture versus a single agent with more tools? . . . 158
Q06.04 How do you handle tool call failures and make agents resilient? . . . 159
Q06.05 An agent with database write access is deployed. What are the security risks and how do you mitigate them? . . . 161
Q06.06 How do you evaluate an agent? Design the eval suite for a coding agent. . . . 164
Q06.07 Your agent's average cost is $0.30 per task but p99 is $12. Explain and fix. 166
Q06.08 Design the memory architecture for a personal assistant agent used daily over years. . . . 168
Q06.09 How would you migrate a rule-based workflow automation system to an agentic one? What stays rules-based? . . . 170
Q06.10 An agent in production suddenly starts failing after a model provider update. Walk through the response. . . . 172
» 6.12 Mock Interview: An IT-Support Agent on MCP . . . 174

[7] Multimodal AI . . . 177
» 7.1 Overview . . . 177
» 7.2 Vision-Language Model Architecture . . . 178
» 7.3 Document Intelligence and OCR . . . 180
» 7.4 Speech: ASR and TTS . . . 181
» 7.5 Image Generation . . . 181
» 7.6 Multimodal RAG . . . 182
» 7.7 Interview Questions and Answers . . . 182
Q07.01 Design a document understanding system for insurance claims: scanned forms, photos of damage, and handwritten notes. . . . 182
Q07.02 Your VLM performs well on benchmarks but fails on your customers' documents. Diagnose. . . . 184
Q07.03 Explain how vision-language models work. What is the connector and why does it matter? . . . 186
Q07.04 Design a real-time voice assistant with under 500ms response latency. . . . 187

Q07.05 Multimodal RAG over technical manuals with diagrams. Users ask questions the diagrams answer. How do you build it? . . . 189
Q07.06 Your VLM-based OCR system reports 96% accuracy but finance says the numbers are wrong. Investigate. . . . 192
Q07.07 How do you handle video understanding when a 10-minute video exceeds any context window? . . . 194
Q07.08 Compare a specialised OCR pipeline against an end-to-end VLM for invoice processing at 100,000 documents/month. . . . 195
Q07.09 Explain diffusion models and classifier-free guidance. How would you productionise image generation at scale? . . . 197
Q07.10 Design content moderation for a user-facing image generation product. . . . 200

[8] Fine-Tuning and Model Adaptation . . . 203

» 8.1 Overview . . . 203
» 8.2 Supervised Fine-Tuning . . . 205
» 8.3 LoRA and QLoRA . . . 206
» 8.4 Preference Optimisation . . . 208
» 8.5 Dataset Preparation . . . 210
» 8.6 Synthetic Data . . . 211
» 8.7 Evaluating a Fine-Tune . . . 211
» 8.8 Interview Questions and Answers . . . 212
Q08.01 When should you fine-tune versus use RAG? Give a decision framework and a case where you need both. . . . 212
Q08.02 Explain LoRA mathematically. Why does it work, and what are the practical hyperparameters? . . . 214
Q08.03 Your fine-tuned model is better at the target task but worse at everything else. Explain and fix. . . . 215
Q08.04 Design a fine-tuning pipeline for a model that must be updated monthly with new data. . . . 217
Q08.05 Compare RLHF, DPO, and KTO. Which would you choose for a product with a thumbs up/down button? . . . 220
Q08.06 You have 500 hand-labelled examples and need 10,000. How do you generate synthetic data safely? . . . 222
Q08.07 How do you serve 50 fine-tuned model variants for 50 customers cost-effectively? . . . 224
Q08.08 Your fine-tune shows 15% improvement on your eval set but users report no difference. What went wrong? . . . 225
Q08.09 Your SFT run's training loss is decreasing smoothly but eval quality is flat or worse. Debug it. . . . 227
Q08.10 How would you distil a frontier model into a 3B model you can serve yourself? Cover data, method, legality, and evaluation. . . . 229

PART IV Production . . . 233

[9] Production Generative AI Systems . . . 234

» 9.1 Overview . . . 234
» 9.2 API Design . . . 235
» 9.3 Guardrails . . . 236
» 9.4 Caching . . . 237
» 9.5 Rate Limiting and Cost Control . . . 238
» 9.6 Observability . . . 239
» 9.7 Deployment . . . 239
» 9.8 Interview Questions and Answers . . . 240
Q09.01 Design the complete production architecture for an enterprise AI assistant serving 50,000 employees. . . . 240
Q09.02 Your LLM costs jumped 300% overnight with no traffic increase. Investigate. 242

Q09.03 Design a guardrail system. What do you check, where, and how do you handle guardrail failures? . . . 244
Q09.04 How do you monitor LLM quality in production when there is no ground truth? 246
Q09.05 Design a semantic caching system. What are the risks and how do you mitigate them? . . . 248
Q09.06 A provider outage takes down your main model. Design the failover strategy. 250
Q09.07 How do you handle PII in an LLM pipeline that must comply with GDPR? . . . 252
Q09.08 Design the deployment and autoscaling strategy for self-hosted LLM inference on Kubernetes. . . . 255
Q09.09 A customer reports the model "stops mid-sentence" intermittently. Walk through the investigation. . . . 257
Q09.10 Design the on-call runbook for a generative AI service. What alerts fire, and what does the responder do? . . . 259

[10] Evaluation, Optimisation, and System Design . . . 262

» 10.1 Overview . . . 262
» 10.2 Evaluation Methods . . . 263
» 10.3 Benchmarks . . . 265
» 10.4 Performance Optimisation . . . 266
» 10.5 Cost Optimisation . . . 267
» 10.6 Hallucination Detection . . . 268
» 10.7 Security and Responsible AI . . . 268
» 10.8 System Design Interview Framework . . . 269
» 10.9 Interview Questions and Answers . . . 269
Q10.01 Design an end-to-end evaluation system for a customer-facing LLM product from scratch. . . . 270
Q10.02 Design a production RAG system for a legal firm: 10 million documents, strict accuracy requirements, full auditability. . . . 272
Q10.03 How do you detect and reduce hallucinations in a production RAG system? . . . 274
Q10.04 Your CFO wants to cut AI spend 50% without hurting quality. Present your plan. . . . 276
Q10.05 Design an AI system that summarises 10,000 documents daily with 99.9% reliability. . . . 278
Q10.06 Compare LLM-as-judge with human evaluation. When is each appropriate, and how do you validate a judge? . . . 280
Q10.07 Walk me through designing an AI coding assistant end to end. . . . 282
Q10.08 What are the most important lessons you would give someone building their first production generative AI system? . . . 284
Q10.09 You are launching a new product with no production traffic. How do you build an evaluation set from a cold start? . . . 285
Q10.10 Design the process for deciding whether to deploy a quantized model. FP16, FP8, INT8, or INT4? . . . 287

Glossary . . . 291

Closing Notes . . . 298

CHEAT SHEET

+==+

[How to use this sheet]

Read this sheet the night before the interview. Each block gives a formula, a worked number, and the rule that you say aloud. The chapter reference tells you where the full explanation is.

+--+

[01 // MEMORY: WEIGHTS]

$$\text{weight bytes} = N \times \text{bytes per parameter}$$

Model	BF16 (2 B)	FP8 (1 B)	NVFP4 (≈0.56 B)
8B	16 GB	8 GB	4.5 GB
70B	140 GB	70 GB	39 GB
405B	810 GB	405 GB	228 GB

Say: “Weights are a fixed cost. The KV cache is the cost that grows with traffic.” Ch. 2, Ch. 4

[02 // MEMORY: KV CACHE]

$$\text{KV bytes} = 2 \times L \times H_{kv} \times d_{\text{head}} \times T \times B \times s$$

Model (GQA-8, FP16)	Per token	One 8k seq.	One 128k seq.
8B (L=32)	128 KB	1.07 GB	17.2 GB
70B (L=80)	320 KB	2.68 GB	42.9 GB
405B (L=126)	504 KB	4.23 GB	67.6 GB

* MHA with h KV heads multiplies these values by h/H_{kv}.
* An FP8 KV cache divides them by 2.
* Concurrency ≈ (GPU memory − weights − activations) / (KV bytes for one sequence).

Say: “Memory, not compute, sets the concurrency.” Ch. 2

[03 // MEMORY: TRAINING AND ADAPTATION]

Method	7B	70B	Rule
Full FT (AdamW, bf16)	≈120 GB	≈1,100 GB	≈16 bytes per parameter, plus activations
LoRA (bf16 base)	≈20 GB	≈170 GB	Frozen base, adapter gradients only
QLoRA (NF4 base)	≈7 GB	≈46 GB	4-bit base, bf16 adapters

LoRA parameters for one matrix $W \in \mathbb{R}^{d \times k}$ at rank r: $r(d+k)$. For $d = k = 4096$ and $r = 16$: 131,072 parameters, which is 0.8% of 16.8M. Set $\alpha = 2r$ as a start value. Ch. 8

[04 // COMPUTE AND THE ROOFLINE]

* Forward pass: $\approx 2N$ FLOPs for each token.
* Training: $\approx 6ND$ FLOPs. Chinchilla optimum: $D \approx 20N$.
* Arithmetic intensity of prefill ≈ prompt length (FLOP per byte, FP16).
* Arithmetic intensity of decode ≈ batch size.
* H100 ridge point: 989 TFLOPS / 3.35 TB/s ≈ 295 FLOP per byte.

$$t_{\text{step}} \geq \frac{\text{weight bytes} + \text{KV bytes read}}{\text{memory bandwidth}}$$

70B decode floor (weights only)	Floor	Note
H100, FP16 (3.35 TB/s)	42 ms	Reference value
B200, FP8 (8 TB/s)	8.8 ms	[2026]
B200, NVFP4 (8 TB/s)	4.9 ms	[2026] verify quality first

Say: “Prefill is compute-bound. Decode is bandwidth-bound. They have different fixes.” Ch. 2, Ch. 4

[05 // LATENCY AND THROUGHPUT]

* TTFT ≈ queue time + prefill time.
* End-to-end latency ≈ TTFT + output tokens × TPOT.
* Little’s law: active requests = arrival rate × time in system.
* Static batch utilisation $= \sum \ell_i / (B \times \max \ell_i)$. Example: 17%.
* Speculative decoding: $E[n] = (1 - a^{k+1})/(1 - a)$ tokens for each target pass. For $a = 0.8$ and $k = 5$: 3.7 tokens. With a drafter $20\times$ less expensive: speedup $\approx 3.7/1.25 \approx 3.0\times$.
* Voice agent: the budget is the sum of endpointing, ASR, LLM TTFT, TTS first audio, and network. All stages must stream.

Say: “A large p50 to p99 spread is queueing or a mixed workload, not the model.” Ch. 4, Ch. 7

[06 // TOKENS AND COST]

Content	Tokens per 1k chars	Rule
English prose	≈250	1 token ≈ 4 characters ≈ 0.75 words
Source code	320–400	Budget $1.5\times$ English
JSON / XML	400–500	Use compact schemas
Chinese, Japanese	600–900	2–$3\times$ English
Base64, hashes, UUIDs	700–1,000	Do not put raw blobs in a prompt

$$\text{cost per request} = T_{\text{in}}\, p_{\text{in}} + T_{\text{out}}\, p_{\text{out}} \quad \text{(reasoning tokens count as output)}$$

* Prompt cache break-even: $n > (w-1)/(1-r)$ reuses. For $w = 1.25$ and $r = 0.1$: one reuse. For $w = 2.0$: two reuses. [2026]
* Few-shot cost for each month $= E \times R \times p/10^6$. 2,000 example tokens × 5M requests × $2.50 = $25,000.

Say: "I count the tokens before I select the model." Ch. 1, Ch. 3, Ch. 4

[07 // RETRIEVAL]

* Reciprocal rank fusion: $\text{RRF}(d) = \sum_r 1/(k + \text{rank}_r(d))$, with $k = 60$.
* Index memory = vectors × dimensions × bytes. 500M × 1024 × 4 B = 2 TB.
* Matryoshka search at 256 dimensions decreases index memory by $4\times$.

Symptom	Where to look first
Low recall@k	Parsing, chunking, the embedding model, hybrid search
High recall, low faithfulness	Generation: prompt, context order, model
High faithfulness, low relevancy	Query intent; the retriever finds the wrong subject
Global question fails	Use structured extraction or GraphRAG, not top-k

Say: "Retrieval errors and generation errors look the same from outside. I measure them separately." Ch. 5

[08 // AGENTS]

$$P(\text{task success}) = p^n \quad \text{for } n \text{ independent steps}$$

Per-step p	10 steps	20 steps	Meaning
0.95	60%	36%	Good demo, bad product
0.99	90%	82%	Usable with retries
0.999	99%	98%	Target for autonomous work

* Cost per successful task = total cost / successful tasks.
* Context cost grows quadratically with steps: 30 steps cost ≈ $9\times$ 10 steps.

* Tool tokens = tools × tokens per tool. 50 × 300 = 15,000 tokens for each request.
* Human approval follows reversibility: reversible actions run, others wait.
* MCP [2026]: stateless, no token passthrough, handles are not identity.

Say: “Buy only the autonomy that the problem makes necessary.” Ch. 6

[09 // EVALUATION]

95% interval $\approx \pm 1.96\sqrt{p(1-p)/n}$

Eval set size n	Interval at p=0.5	Meaning
100	±9.8 points	Finds only large regressions
500	±4.4 points	Release gate for one slice
2,000	±2.2 points	Detects changes of approximately 3 points

* Calibrate an LLM judge against human labels before you trust it.
* Control position bias: swap the order of A and B and score both orders.
* Report each slice. An aggregate can hide a regression in one slice.

Say: “A half-point change in the aggregate tells me nothing without an interval.” Ch. 10

[10 // CAPACITY PLANNING CHAIN]

[1] Daily active users × requests for each user each day ÷ 86,400 = average requests per second.
[2] Multiply by the peak factor (typically 3 to 5).
[3] Multiply by output tokens for each request = output tokens per second.
[4] Divide by the measured tokens per second of one replica at the latency target.
[5] Multiply by 1.4 for headroom = replicas. Multiply by GPUs for each replica.

Example: 1M users × 10 requests ÷ 86,400 × 4 ≈ 463 requests per second at peak. Ch. 10

[11 // DECISION TABLES]

If the problem is	Then select
Missing or changing facts	RAG
Wrong form, tone, or format	Prompting first, then LoRA
A stable task at high volume	Fine-tune or distill a small model
The full document must be in view	Long context, if cost permits
Citations, access control, deletion	RAG, not long context
Strict output structure	Constrained decoding, not a prompt request
Memory-bound decode	Quantization (FP8, then FP4) and GQA
Shared long prefixes	Prefix caching or provider prompt caching
Hard, verifiable problems	A reasoning model with a token budget
Easy, high-volume traffic	A small model behind a router

Adaptation step	Typical time, cost, and risk
1. Prompting	Hours; $0; you can reverse it at once
2. Few-shot and retrieval	Days; a token cost for each call; you can reverse it
3. LoRA / QLoRA	Days; $50–500; you can change the adapter
4. Full fine-tune	Weeks; $1k–50k; a new model to serve
5. Continued pretraining	Months; $100k or more; a research program

Ch. 3, Ch. 5, Ch. 8

[12 // SYSTEM DESIGN ANSWER ORDER]

[1] Clarify scale, latency target, quality bar, budget, data sensitivity, and build or buy (5 minutes).
[2] Define success and the metric that measures it.
[3] Draw the architecture: gateway, guardrails, context, router, inference, evaluation.
[4] Go deep on the hardest part.
[5] Do the arithmetic aloud: tokens, GPUs, cost for each request.
[6] Name the failure modes, how you detect them, and how the system degrades.
[7] Say what you build first and what you defer.

Ch. 10

```
8888888
  888
  888
  888
  888
  888
  888
8888888
```

FOUNDATIONS

+==+

Before a model can answer a question, the question has to become tokens and the tokens have to become computation. This part covers that conversion and its price: what generative modelling actually estimates, why the Transformer displaced everything before it, what a token costs in each language and each modality, and how every architectural choice in a modern LLM — rotary positions, grouped-query attention, sparse experts, fused attention kernels — is an answer to a memory or bandwidth problem rather than an intelligence one.

```
 .d8888b.  d888
d88P  Y88bd8888
888    888  888
888    888  888
888    888  888
888    888  888
Y88b  d88P  888
 "Y8888P" 8888888
```

FOUNDATIONS OF GENERATIVE AI

+==========+

subscribe -> aiengineeringinsider.substack.com/subscribe

[CHAPTER BRIEF]

What a generative model actually estimates, why the Transformer won, and the three numbers—tokens, parameters, and context—that price every system you will ever design on top of one.

+----------+

» 1.1 OVERVIEW

.----------.

Almost every production incident in a generative AI system traces back to a misunderstanding at this level. A team blames the model for truncating an answer when the real cause is that their tokenizer expanded a Japanese document to three times the token count they budgeted. Another team pays for a 70B model when the task is extraction and an 8B model with a good prompt would have matched it. A third ships a retrieval system whose embedding model was never trained on their domain vocabulary and wonders why recall is poor.

This chapter builds the mental model that prevents those failures. We define what generative modelling *is* in probabilistic terms, walk the Transformer components that make it computationally tractable, and then work through the four quantities that dominate every downstream design decision: how text becomes tokens, how tokens become vectors, how model capability scales with compute, and where the whole approach breaks down.

[CORE DEFINITION]

A **generative model** learns to approximate the joint probability distribution $p(x)$ over a data space, so that it can *sample* new points from that space. A **discriminative model** learns only the conditional $p(y \mid x)$—a decision boundary. Autoregressive language models factor the joint distribution using the chain rule of probability:

$$p(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}),$$

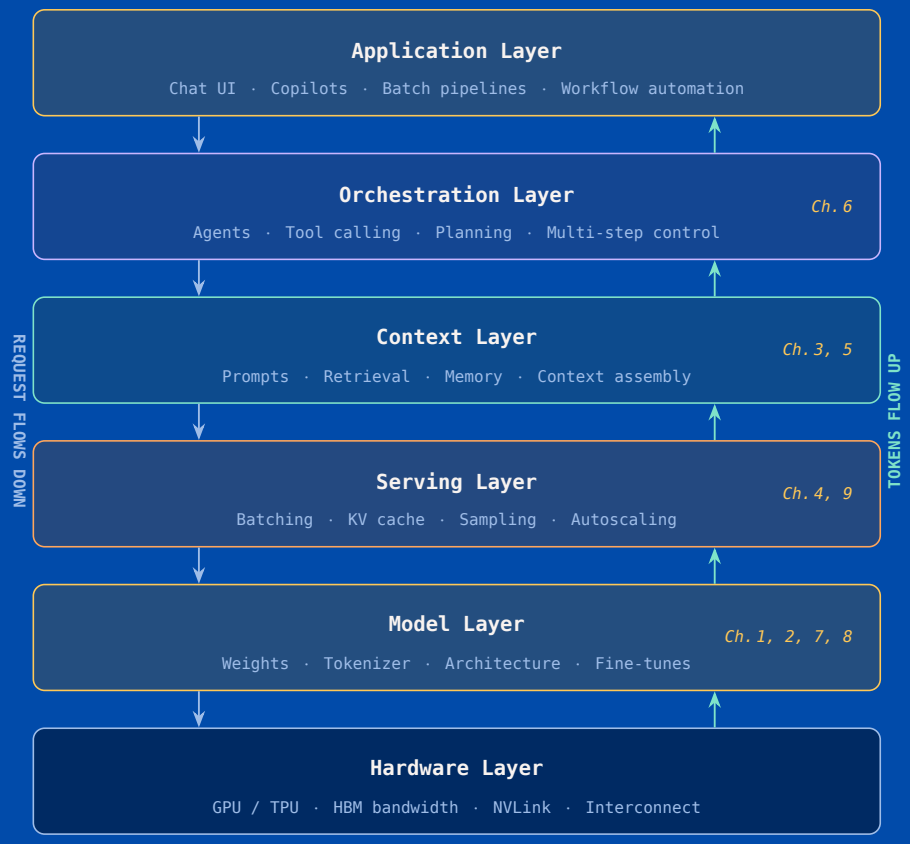
which reduces "model all of language" to "predict the next token, repeatedly." Every capability an LLM exhibits—reasoning, translation, code synthesis—is an emergent consequence of optimising that one objective at scale.

[* KEY INSIGHT]

The chain-rule factorisation is why LLMs are so general and why they hallucinate. The model is never trained to be *correct*; it is trained to be *likely*. A fluent falsehood and a fluent truth receive identical treatment from the loss function unless the training data disambiguates them. Every grounding technique in this book—RAG, tool use, verification—exists to inject a correctness signal that pretraining does not provide.

-> 1.1.1 The Generative AI Stack

Before the details, it helps to see where each chapter of this book sits. Figure 1.1 shows the layered architecture that every serious generative AI product converges on, regardless of vendor.



[FIG 1.1] The generative AI stack. Each layer consumes the guarantees of the layer beneath it and exposes a narrower, safer contract upward. Chapter numbers on the right mark where each layer is covered in depth.

» 1.2 DISCRIMINATIVE VERSUS GENERATIVE MODELS

The distinction is not academic—it changes what you can build, how you evaluate, and what failure looks like.

A discriminative classifier trained to label support tickets as `billing` or `technical` learns a boundary in feature space. It cannot write a ticket, cannot explain its decision beyond feature attribution, and degrades predictably when the input distribution shifts. Its failure mode is *misclassification*, and it is measurable with a confusion matrix.

A generative model trained on the same corpus learns the shape of the entire distribution. It can classify (by conditioning), summarise, rewrite, translate, and extract—all without task-specific heads. Its failure mode is *plausible fabrication*, which no confusion matrix detects.

Dimension	Discriminative	Generative
Learns	$p(y \mid x)$ — a decision boundary	$p(x)$ or $p(x \mid c)$ — the data distribution
Typical models	Logistic regression, XGBoost, BERT classifiers, CNNs	GPT-family, Llama, Claude, diffusion models, VAEs
Data hunger	Modest; thousands of labelled examples	Extreme; trillions of unlabelled tokens
Task transfer	Retrain per task	Prompt or lightly fine-tune per task
Failure mode	Misclassification (detectable)	Hallucination (fluent, undetectable without grounding)
Evaluation	Accuracy, F1, AUC	Perplexity, human preference, task-specific rubrics, LLM-as-judge
Inference cost	Microseconds, CPU-friendly	Milliseconds to seconds per token, GPU-bound
Use when	Labels are cheap, latency is tight, task is fixed	Task is open-ended, labels are scarce, output is text

> **» FROM THE FIELD**
>
> The most common architecture mistake I see in 2026 is using a 70B parameter generative model to do a job a 400M parameter encoder does better, faster, and for 1/200th the cost. Intent classification, PII detection, toxicity screening, and semantic routing are all discriminative problems. Use a fine-tuned ModernBERT or a small cross-encoder. Save the generative budget for the parts of the pipeline that actually need to *write* something.

» 1.3 FOUNDATION MODELS

A **foundation model** is a model trained on broad data at scale, using self-supervision, that is designed to be adapted rather than used directly (Bommasani et al., 2021). Three properties define the category.

Scale Trained on corpora measured in trillions of tokens with compute budgets measured in exaFLOP-days. The scale is what produces generality.

Self-supervision The training signal comes from the data itself—next-token prediction, masked-token reconstruction, or noise removal—so no human labelling bottleneck constrains corpus size.

Adaptability A single set of weights serves hundreds of downstream tasks via prompting, retrieval augmentation, adapters, or full fine-tuning. This is the economic argument: amortise one enormous training run across every application.

Foundation models come in three architectural families, and the interview question "which would you choose and why?" is answered by matching the family to the information flow the task requires.

Family	Objective	Best at	Examples
Decoder-only	Next-token prediction, causal mask	Open-ended generation, chat, code, agentic control	GPT-4o, Claude, Llama 3.x, Qwen, DeepSeek-V3
Encoder-only	Masked-token prediction, bidirectional	Classification, embeddings, reranking, extraction	BERT, RoBERTa, ModernBERT, DeBERTa
Encoder–decoder	Denoising / span corruption, seq2seq	Translation, summarisation with a fixed source	T5, FLAN-T5, BART, mT5
Diffusion	Iterative denoising of latents	Images, audio, video; high-fidelity continuous data	Stable Diffusion, Flux, SDXL

» 1.4 NEURAL NETWORK FUNDAMENTALS FOR THE TRANSFORMER

You do not need to derive backpropagation in an interview, but you do need to explain *why* each Transformer component exists. Each one solves a specific failure of the architecture that preceded it.

-> 1.4.1 Self-Attention: The Core Intuition

Recurrent networks processed tokens sequentially, which meant that information from token 1 reached token 500 only by surviving 499 lossy state updates. Long-range dependencies decayed. Training could not be parallelised across the sequence.

Self-attention replaces sequential state passing with direct, learned, all-pairs communication. Every token emits three vectors:

Query (Q) "What information am I looking for?"
Key (K) "What information do I offer?"
Value (V) "Here is the content I contribute if you attend to me."

The output for each position is a weighted average of all value vectors, where the weights come from query–key similarity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Two details carry interview weight. The $\sqrt{d_k}$ divisor prevents the dot products from growing with dimension and saturating the softmax into a near-one-hot distribution with vanishing gradients (Vaswani et al., 2017). The mask M sets future positions to $-\infty$ in decoder-only models, enforcing causality so that position t cannot see $t+1$—which is exactly what makes the KV cache of Chapter 2 possible.

[! PITFALL / WARNING]

Self-attention is $O(T^2 d)$ in time and, if materialised naively, $O(T^2)$ in memory. At $T = 128{,}000$ the attention matrix alone would be roughly 16 billion entries per head per layer. This is why FlashAttention (which never materialises the matrix) is not an optimisation you may skip—it is a precondition for long context. See Section 2.7.

-> 1.4.2 Multi-Head Attention

A single attention operation computes one weighted average, which forces the model to blend all relational signals into one distribution. Multi-head attention splits d_{model} into h subspaces, runs attention independently in each, and concatenates. Empirically, different heads specialise: some track syntactic dependencies, some track coreference, some perform the induction (copy-the-pattern) behaviour that underlies in-context learning.

[SCALED DOT-PRODUCT ATTENTION — THE REFERENCE IMPLEMENTATION]

```python
import torch
import torch.nn.functional as F

def multi_head_attention(x, w_qkv, w_out, n_heads, causal=True):
    """x: (batch, seq, d_model). Returns (batch, seq, d_model)."""
    B, T, D = x.shape
    head_dim = D // n_heads

    # One fused projection is cheaper than three separate matmuls.
    qkv = x @ w_qkv                          # (B, T, 3D)
    q, k, v = qkv.chunk(3, dim=-1)

    # (B, n_heads, T, head_dim) -- heads become a batch dimension.
    reshape = lambda t: t.view(B, T, n_heads, head_dim).transpose(1, 2)
    q, k, v = reshape(q), reshape(k), reshape(v)

    scores = (q @ k.transpose(-2, -1)) / (head_dim ** 0.5)   # (B, H, T, T)

    if causal:
        mask = torch.triu(torch.ones(T, T, device=x.device), diagonal=1).bool()
        scores = scores.masked_fill(mask, float("-inf"))

    out = F.softmax(scores, dim=-1) @ v                      # (B, H, T, head_dim)
    out = out.transpose(1, 2).reshape(B, T, D)
    return out @ w_out
```

-> 1.4.3 Feed-Forward Networks

Attention moves information *between* positions; it performs no nonlinear computation *within* a position. The position-wise feed-forward network supplies that:

$$\text{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2,$$

typically with an inner dimension of $4d_{\text{model}}$ and a gated activation (SwiGLU in most modern models). This block holds roughly two-thirds of a dense Transformer's parameters, which is why Mixture-of-Experts targets it for sparse activation (Section 2.6).

-> 1.4.4 Residual Connections and Layer Normalisation

Residual connections ($x + f(x)$) give gradients an identity path to every layer, which is what makes 80- and 120-layer stacks trainable at all (He et al., 2016). Layer normalisation stabilises activation statistics per token.

The placement matters and is a favourite interview probe. **Post-LN** (the original 2017 design) normalises after the residual add and requires learning-rate warmup to avoid divergence. **Pre-LN** normalises the branch input, keeps the residual stream clean, and trains stably without careful warmup—so every modern large model uses it. RMSNorm (Zhang & Sennrich, 2019) drops the mean-centring term, saving compute for no measurable quality loss, and is now the default in Llama, Qwen, Mistral, and DeepSeek.

» 1.5 TOKENIZATION

Tokenization is the single most under-appreciated source of production bugs in generative AI. It determines your cost, your effective context length, your multilingual fairness, and a surprising number of your model's reasoning failures.

[TOKENIZATION]

The reversible mapping from a text string to a sequence of integer IDs drawn from a fixed vocabulary. Subword algorithms sit between two bad extremes: character tokens produce sequences too long to attend over, and word tokens produce vocabularies too large to embed and cannot represent unseen words.

-> 1.5.1 The Three Algorithms

Byte-Pair Encoding (BPE) starts with individual bytes and greedily merges the most frequent adjacent pair, repeating until the vocabulary target is reached (Sennrich et al., 2016). **Byte-level BPE**—used by GPT-family models and Llama—operates on raw UTF-8 bytes, which guarantees that no input is ever out-of-vocabulary, including emoji, Cyrillic, and corrupted encodings.

WordPiece (BERT) merges the pair that maximises corpus likelihood rather than raw frequency, a subtly different criterion that tends to produce more linguistically coherent subwords.

SentencePiece is not a merge algorithm but a *framework* that treats the input as a raw Unicode stream—spaces included, encoded as `U+2581`—so it needs no language-specific pre-tokenizer. It can run either BPE or the Unigram algorithm underneath. This is what makes it the standard for multilingual and non-space-delimited languages such as Japanese and Thai.

Algorithm	Merge criterion	Properties	Used by
BPE	Highest pair frequency	Fast, deterministic, simple; can split words oddly at boundaries	GPT-2, RoBERTa
Byte-level BPE	Frequency over UTF-8 bytes	Zero OOV by construction; inflates token count for non-Latin scripts	GPT-4o, Llama 3, Mistral
WordPiece	Max corpus likelihood gain	Cleaner morphological splits; `##` continuation marker	BERT, DistilBERT, ELECTRA
Unigram (SentencePiece)	Prune tokens by likelihood loss	Probabilistic, supports subword regularisation sampling	T5, ALBERT, XLNet

-> 1.5.2 Token Counting and Its Consequences

For English prose the working heuristic is 1 token ≈ 4 characters ≈ 0.75 words. That heuristic is dangerously wrong for everything else.

Content type	Tokens per 1k chars	Engineering consequence
English prose	≈ 250	Baseline for all capacity planning
Source code	≈ 320–400	Indentation and punctuation tokenize poorly; budget 1.5×
JSON / XML payloads	≈ 400–500	Structural characters dominate; prefer compact schemas
CJK (Chinese, Japanese)	≈ 600–900	Same document costs 2–3× more than its English translation
Base64 / hashes / UUIDs	≈ 700–1000	Near-worst case; never put raw blobs in a prompt

[AUDITING REAL TOKEN COST BEFORE YOU COMMIT TO A CONTEXT BUDGET] ----------

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")   # GPT-4o family

def audit(label: str, text: str, price_per_mtok: float = 2.50) -> dict:
    ids = enc.encode(text)
    n = len(ids)
    return {
        "label": label,
        "chars": len(text),
        "tokens": n,
        "chars_per_token": round(len(text) / max(n, 1), 2),
        "usd_per_1k_calls": round(n / 1_000_000 * price_per_mtok * 1000, 4),
    }

samples = {
    "english":  "The quick brown fox jumps over the lazy dog. " * 20,
```

```
    # "He learns a new technology every day." in Japanese, written with
    # \u escapes so this listing stays ASCII. In practice, load real locale
    # samples from a fixtures file rather than embedding them in source.
    "japanese": ("\u5f7c\u306f\u6bce\u65e5\u65b0\u3057\u3044"
                 "\u6280\u8853\u3092\u5b66\u3093\u3067\u3044\u307e\u3059\u3002") * 20,
    "json":     '{"user_id": "a3f9-22b1", "status": "active"}, ' * 20,
}

for label, text in samples.items():
    print(audit(label, text))

# Typical result: Japanese costs ~2.4x more tokens than English for the
# same *information*, and JSON costs ~1.7x more than the prose equivalent.
# Both facts belong in your capacity model, not in a post-mortem.
```

[! PITFALL / WARNING]

Never estimate tokens with `len(text.split())` or a character divisor in production. Count with the actual tokenizer for the actual model. A pricing model built on `chars / 4` under-reports Japanese traffic by more than half; the first month with real international users turns that into a budget overrun and a wave of context-overflow errors.

-> 1.5.3 Context Window

The **context window** is the maximum number of tokens the model can attend over in a single forward pass—prompt plus generated output together. It is a hard architectural limit, not a soft preference.

Three practical consequences that interviewers probe:

1. **Advertised is not usable.** A 128k window must also hold your system prompt, few-shot examples, retrieved chunks, tool schemas, conversation history, *and* the reserved output budget. Plan for roughly 70–80% usable.
2. **Quality degrades before capacity does.** Models retrieve information placed at the beginning and end of a long context far more reliably than information in the middle—the "lost in the middle" effect (Liu et al., 2024). Put the critical instruction last.
3. **Cost is superlinear in prefill.** Attention is quadratic in sequence length, so doubling the prompt more than doubles time-to-first-token. Long context is not a substitute for retrieval; it is a more expensive alternative to it.

» 1.6 EMBEDDINGS

An **embedding** is a dense vector that places an input in a geometric space where distance encodes semantic relatedness. Two distinct kinds appear in a production system and conflating them is a classic interview trap.

Token embeddings are the model's internal lookup table—one learnable vector per

vocabulary entry, retrieved at the input layer and refined by every subsequent Transformer block. They are contextual only after the first layer.

Sentence or document embeddings are the output of a dedicated model (usually an encoder trained with a contrastive objective) that maps a whole passage to a single vector for similarity search. These are what your vector database stores.

Embedding type	Produced by	Used for
Token embedding	Model input layer	Internal representation; never queried directly
Semantic (text) embedding	Contrastive encoder	Vector search, clustering, deduplication, classification
Cross-modal embedding	CLIP / SigLIP-style dual encoder	Text-to-image retrieval, zero-shot classification
Late-interaction (multi-vector)	ColBERT / ColPali	High-recall retrieval; one vector per token, higher index cost

[* KEY INSIGHT]

Cosine similarity measures direction, not truth. Two passages that both discuss "insurance claim denial" will sit close together whether one *approves* the claim and the other *denies* it. Dense retrieval is a recall mechanism, not a precision mechanism—which is precisely why the reranking stage of Section 5.5 is non-optional in production RAG.

» 1.7 SCALING LAWS AND EMERGENT ABILITIES

Kaplan et al. (2020) showed that test loss falls as a smooth power law in parameters N, dataset size D, and compute C—smooth enough that a lab can predict a large model's loss from small-scale runs before spending the training budget.

Hoffmann et al. (2022), the "Chinchilla" work, corrected the optimal allocation: for a fixed compute budget, parameters and training tokens should scale roughly *equally*, implying about 20 tokens per parameter. Most models before that paper were badly under-trained.

> **» FROM THE FIELD**
>
> Chinchilla optimality is a *training-compute* criterion, and it is the wrong objective if you are going to serve the model. A model trained far past Chinchilla—Llama 3 8B saw roughly 15 trillion tokens, about 1875 tokens per parameter—costs more to train but is dramatically cheaper to *serve* at its quality level. When the model runs for a year in production, inference cost dominates the total bill. This is why the industry moved to deliberately "over-training" small models.

Emergent abilities are capabilities absent in smaller models that appear, seemingly discontinuously, past a scale threshold: multi-step arithmetic, instruc-

tion following, chain-of-thought reasoning, in-context learning from a handful of examples. Wei et al. (2022) documented the phenomenon; Schaeffer et al. (2023) argued convincingly that much of the apparent discontinuity is an artefact of using all-or-nothing metrics like exact-match accuracy, and that continuous metrics reveal smooth improvement. Being able to state both sides of this is a strong signal in a senior interview.

» 1.8 THE ECOSYSTEM: OPEN VERSUS CLOSED

Provider	Access model	Characteristic strengths and design notes
OpenAI	Closed API	Broad tool/function-calling ecosystem, structured outputs, strong multimodal and reasoning tiers
Anthropic	Closed API	Long-context reliability, constitutional-AI safety training, strong agentic tool use and computer use
Google	Closed API	Very long context, native multimodality across text/image/audio/video, TPU-backed economics
Meta (Llama)	Open weights	The de facto self-hosting baseline; the largest fine-tuning and tooling ecosystem
DeepSeek	Open weights	MoE efficiency (MLA + fine-grained experts), reasoning distillation, aggressive cost per token
Qwen	Open weights	Strong multilingual coverage, dense and MoE variants across many sizes, permissive licensing
Mistral	Mixed	Efficient dense and sparse models, European data-residency positioning

[! PITFALL / WARNING]

Model names, prices, context limits, and benchmark scores change monthly. Treat every specific figure in this book as a snapshot for reasoning about *trade-offs*, and verify current numbers against the provider's own documentation before you put them in a design document.

-> 1.8.1 The Build-versus-Buy Decision

Factor	Closed API	Open weights, self-hosted
Time to first demo	Hours	Days to weeks
Unit economics	Linear in tokens; cheap at low volume	Fixed GPU cost; cheaper past a crossover point
Data residency	Vendor-controlled; contractual guarantees	Full control; required for many regulated workloads
Customisation	Prompting, limited fine-tuning	Full fine-tuning, LoRA, quantization, custom kernels
Latency floor	Network round trip plus queueing	Controllable; co-locate with your data
Version stability	Vendor may deprecate or silently update	You pin the checkpoint forever
Ops burden	Near zero	GPU capacity, autoscaling, upgrades, on-call

» BEST PRACTICE **[ok]**

Start on a closed API to validate that the product works at all. Instrument token volume from day one. Compute the crossover point honestly—including the engineer-months of serving work, not just GPU rental—and migrate the highest-volume, lowest-complexity route first. Hybrid routing (Section 10.5) usually beats an all-or-nothing decision.

» 1.9 CAPABILITIES AND LIMITATIONS

An honest account of the limits is what separates a senior candidate from a enthusiastic one.

Hallucination The model optimises likelihood, not truth. Mitigate with retrieval grounding, citation requirements, and verification passes—never with "please be accurate" in the prompt.

Knowledge cutoff Parametric knowledge is frozen at training time. Anything time-sensitive must arrive through retrieval or tools.

No native arithmetic Numbers are tokens. Multi-digit arithmetic is pattern matching over token sequences and fails unpredictably. Route calculation to a code interpreter.

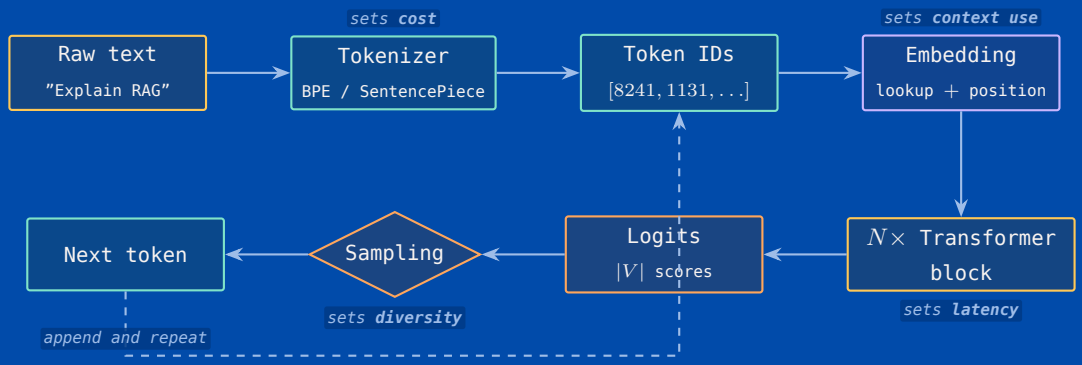
Context degradation Effective attention over very long contexts is worse than nominal capacity implies; position matters.

Non-determinism Even at temperature 0, floating-point non-associativity under varying batch composition makes outputs non-bitwise-reproducible on GPU.

Prompt injection There is no reliable in-band separation between instructions and data. Any text the model reads is a potential instruction (Section 3.8).

Cost and latency Generation is memory-bandwidth-bound and sequential. You cannot

parallelise your way out of a long output.



[FIG 1.2] From raw text to sampled token—the full forward path, with the four quantities that dominate cost annotated at the stage where they are set.

» 1.10 INTERVIEW QUESTIONS AND ANSWERS

Q01.01 **Explain the difference between discriminative and generative models, and describe a production system where you would deliberately use both.**

A > **The distinction.** A discriminative model estimates $p(y \mid x)$: it learns a boundary that separates classes and cannot produce new samples. A generative model estimates $p(x)$ or $p(x \mid c)$: it learns the data distribution well enough to sample from it. Autoregressive LLMs are generative models that factor the joint distribution via the chain rule, $p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$.

The production system. Consider a customer support automation platform handling 2 million tickets a month. A pure-LLM design—send every ticket to a frontier model—is both slow and expensive, and it applies a 10^{11}-parameter generative model to problems that a 10^{8}-parameter classifier solves better. The correct architecture is a cascade:

1. **Discriminative router** (fine-tuned ModernBERT, ≈150M params, CPU-servable, p99 under 15 ms): classify intent into one of 40 categories and predict a complexity score. About 55% of tickets are routine and resolvable by a deterministic template.
2. **Discriminative safety gate** (small cross-encoder): PII detection and toxicity screening. This must be a separate model—you cannot ask the generative model to police its own input reliably.
3. **Generative tier** for the remaining 45%: an LLM with RAG over the knowledge base drafts the response.
4. **Discriminative verifier**: an NLI cross-encoder scores whether the draft is entailed by the retrieved evidence. Low-entailment drafts escalate to a human.

Why this is the right answer. The two model families have complementary fail-